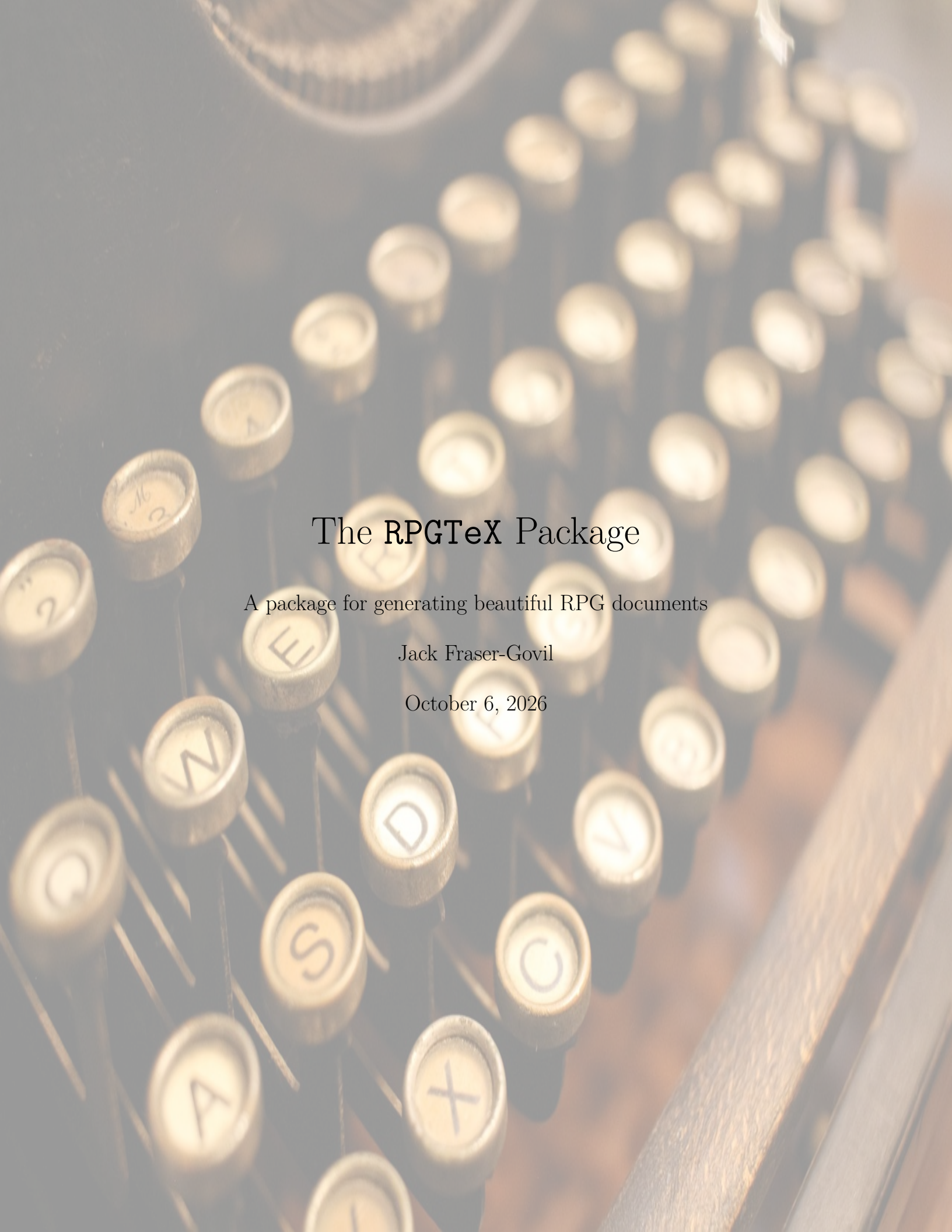




The **RPGTeX** Package

A package for generating beautiful RPG documents

Jack Fraser-Govil

October 6, 2026

Welcome to the *RPGTeX* package. This $\text{\LaTeX}$ package is designed to allow users to flexibly typeset documents associated with Role Playing Games such as *Dungeons & Dragons* -- and many more besides. This package defines a central engine: `rpgcore` which define a number of useful functions and classes, and a flexible set of `themes` which control how those commands are rendered in the final document.

Getting Started

This document is the full reference manual. For a quick example of how to get started with your first *RPGTeX* document, see the [example documents](#), which demonstrate the basic features of this package in a readable fashion.

Attribution & License

This package would not have been possible without the team who developed [its predecessor, the 'DND 5e LaTeX Template'](#). That code was released under an MIT license, the text of which can be found in the `LICENSE` file. `rpgtex` is released under the same license.

AI Usage Declaration

The author of this package used a handful of AI chat tools over the course of the development to help guide the thoughts of the human author. No automated coding tools were used anywhere in this package, and all code was written by hand. All mistakes and bad decisions are organic in nature.

Contents

1: Installation & Usage

Getting RPGTeX	1
Configuring RPGTeX	2
Package & Class Usage	2
Compiling rpgtex Documents	3

Part I: rpgtex For Users

2: Options & Variables

Package Options	6
Colo(u)rs.....	8
Section Names.....	9
Command Line Interface.....	10

Part II: rpgtex For Designers

Chapter 1: Installation & Usage

Getting R_PG_TE_X

There are a number of different ways to acquire R_PG_TE_X. Once you have installed it, it is vital to ensure that it is properly configured (see below).

Direct Installation

The simplest way to use R_PG_TE_X is to install it on the `texmf` path, where the compiler can automatically find it:

bash

```
mkdir -p "$(kpsewhich -var-value TEXMFHOME)/tex/latex" %make sure the directory exists
git clone https://github.com/DrFraserGovil/rpgtex.git "$(kpsewhich -var-value
    TEXMFHOME)/tex/latex/rpgtex"
```

This will clone the repository into your L^AT_EX path.

Indirect Installation

If you want to tinker with R_PG_TE_X -- such as by creating a new theme -- it is helpful to have it in a more accessible location. Clone the repository into a location of your choice:

bash

```
git clone https://github.com/DrFraserGovil/rpgtex.git /your/rpgtex/directory
```

You then have two options to make the package visible to the compiler:

Use TEXINPUTS

Setting the environment variable `TEXINPUTS` allows the compiler access:

bash

```
TEXINPUTS=~ /your/rpgtex/directory/::
```

(Or similar commands, depending on your shell).

Use Symlinks

You can symlink the install location to the `texmf` directory, allowing the compiler to act as if you had performed the `texmf` installation:

bash

```
mkdir -p "$(kpsewhich -var-value TEXMFHOME)/tex/latex"
ln -sf ~/your/rpgtex/directory "$(kpsewhich -var-value TEXMFHOME)/tex/latex/rpgtex"
```

Overleaf (Not recommended!)

We do not recommend using Overleaf since the free-tier subscription has reduced compilation times drastically, making compiling documents using complex packages such as this one extremely difficult. Nevertheless:

1. Download this GitHub repository as a ZIP archive using the Clone or download link above.

2. On Overleaf, click the New Project button and select Upload Project. Upload the ZIP archive you downloaded from this repository.
3. Manually create the file `rpg-config.cfg` with the contents `\edef\RpgPackagePath{}`. This replaces the configuration step described below.

Configuring RPGTeX

Because RPGTeX comes packaged with some resources (namely some fonts) for which TeX does not support relative-pathfinding, it is necessary to provide RPGTeX with a value for the `\RpgPackagePath`.

`\RpgPackagePath {}` (Variable)

A variable holding the absolute path to the root directory of the rpgtex install location.

We provide a number of different ways to set this variable:

1. The provided `rpglatex` compiler automatically inserts a definition based on a `kpsewhich`, using the Command Line Interface (page 10).
2. If the variable does not exist, the RPGTeX engine itself tries the following
 - (a) Searches for the file `core/rpg-config.cfg`, which (hopefully) contains a definition¹, and `\inputs` it.
 - (b) If no file is found, RPGTeX used the `\write18` utility to dispatch its own calls to `\kpsewhich` and build a config file in the expected location. This requires the compiler be run with `-shell-escape` active
 - (c) If this doesn't work, the compiler exits, and suggests that the user manually creates the config file:

```
cd <rpgtex root directory>
cmd="\edef\RpgPackagePath{ $(pwd) }"
echo $cmd >> core/rpg-config.cfg
```

3. We provide a shell script which executes these commands for the user.

The extra configuration steps need only be run once -- once the config file exists, the package will be able to configure itself.

Why is configuration necessary?

TeX is generally set up so that when a file calls `include` or `input` it is possible to use filepaths relative to the package itself. `package.sty` can call `\input{dir/subpackage.tex}` and it will know to first check for the file relative to `package.sty`; even if the package resides within the texmf path and the user has no idea where `package.sty` is.

An annoying exception to this is fonts and typefaces. `xelatex` searches for fonts based on *filepaths relative to the current working directory* -- or from those installed in as system fonts.

Since RPGTeX includes several (license free) typefaces as part of the provided themes, this poses a problem. We must either require that:

1. RPGTeX documents can only be prepared in restricted locations relative to the install location of RPGTeX.
2. Users must identify and specify the RPGTeX root path when preparing a document
3. Users must install the provided fonts to the system path
4. RPGTeX must be configured to know 'where it is', and so provide an absolute filepath to the internal fonts.

The Configuration step is the most portable and easiest-to-use of these options.

Package & Class Usage

RPGTeX can be used either as a standalone package, or as part of a number of classes

¹In the form of `\edef\RpgPackagePath{ path/to/package }`

Standalone Package

The standalone package can be used directly by including the `RPGTeX` package:

```
\documentclass{arbitrary-class}
\usepackage[options]{rpgtex}
```

This will load only the core commands into the document, and (unless called explicitly) no themes will be imported. Using the package in this way does not activate any of the commands which change the overall geometry, background or headers of the document.

Classes

RPGTeX can also be loaded through a number of classes which drastically alter the appearance of the document, defining new geometries backgrounds and adding headers.

The provided classes are:

1. `rpgbook` (page ??). Based on the standard book class, this is designed for larger RPG documents.
2. `rpghandout` (page ??). Based on the article class, this is designed for shorter documents
3. `rpgcard` (page ??). A small-document class designed for creating modular 'handout' cards for items, spells or abilities.

Compiling rpgtex Documents

RPGTeX uses the `fontspec` package to allow custom fonts, and therefore requires compiling with `xelatex` or `lualatex`:

```
xelatex main.tex #works
lualatex main.tex #works
pdflatex main.tex #fails if fonts option passed
```

So long as RPGTeX is on the user's latex path, and the package properly configured (page 2) no further compilation steps are required. However, for ease of use, we provide the `rpglatex` compiler as part of the RPGTeX distribution.

The rpglatex compiler

RPGTeX is shipped with a special compiler, `rpglatex`. This is simply a python3 script which acts as a wrapper around either `xelatex` or `luatex`, but includes several quality-of-life changes to the interface to make it easier to use with `rpgtex`.

```
rpglatex [options] <file>
```

`rpglatex` has the following features:

Feature	Description	Options
Compiler Selection	The <code>xelatex</code> compiler is selected by default, but the <code>-l</code> , <code>-luatex</code> flags set it to use <code>luatex</code> instead.	<code>-l</code> , <code>-luatex</code>
Build Directory	Compilation files (<code>.aux</code> , <code>.log</code> etc.) are stored in a build directory. The default is <code>.build</code> in the calling location, but can be changed with the <code>-b</code> flag	<code>-b <build dir></code>
Output Naming	The name of the output file can be changed from the default (equal to the input tex name)	<code>-o <output name></code>
Multi-pass Compiling	By default, the compiler runs twice in a row to enable references and <code>tikz[remember]</code> commands to function. A full three-compilation suite (necessary for very complex or reference-heavy documents) can be activated with the <code>-f</code> , <code>-full</code> flag	<code>-f</code> , <code>-full</code>

Volume Control	latex is notoriously noisy, producing copious output. By default, this is suppressed and only a summary is printed. The summary can be removed (rendering it completely silent) with the <code>-q</code> command, or the original output recovered in verbose mode; <code>-v</code>. These outputs are always overridden if a compilation error occurs, in which case the full trace is output to the console.	<code>-q, -v</code>
Auto-bibtex	If the <code>-r</code> or <code>-ref</code> flag is set, <code>bibtex</code> is automatically called in between the multi-compilation steps	<code>-r, -ref</code>
Auto-visualisation	If the <code>-show 1</code> option is set (which it is by default), the compiler will attempt to open the output file upon completion of the compilation; automatically opening or context-switching to the document. This can be turned off by calling <code>-show 0</code>	<code>-show</code>
Print Mode	A special interface for <code>RPGTeX</code> , this uses the <code>\RpgCMD</code> interface (page 10) to inject code into the latex document, setting the <code>bg=print</code> mode and suppressing the background output.	<code>-p, -print</code>

PART I

rpgtex For Users

Chapter 2: Options & Variables

RPGTeX defines many dozens to hundreds of variables, most with the (expl3) syntax `\__rpg_[x]`. Most of these are used in the internal functioning of the macros, however a number of them are useful for a user to understand.

Package Options

Whether RPGTeX is loaded directly as a package or indirectly through one of its classes, options may be passed to it which change the behaviour of the resulting document:

```
\documentclass[<options>]{rpgbook}  
\usepackage[<options>]{rpgtex}
```

Each option takes one of two forms:

Key-value options: The name of the option is followed by an equals sign and a value:

```
\usepackage[columns = 2, bg = print]{rpgtex}
```

Flags: Only the name of the option is provided. Its presence enables the associated behaviour:

```
\usepackage[openright]{rpgtex}
```

The available options are:

background <flag>

If present, activates the 'paper background' modules, including the background colour, paper images, geometry overloads and footer changes.

This flag is set by default when loading a class, but must be passed explicitly to enable these modules when loading the package directly.

bg = full / print / none

Controls the behaviour of the background and footer decorations. The value must be one of the three listed above, else an error is thrown. The default value is **full**.

The most visible effect of this option is to control whether the 'background paper' set by `\RpgThemeSetPaper` (page ??) and `\RpgThemeBackgroundColor` (page ??), and the footer decorations set by `\RpgThemeFooterDecoration` (page ??), appear in the document:

Value	Paper	Footer Decorations
full	✓	✓
print	✗	✓
none	✗	✗

This option also changes the behaviour of the `rpgcard` class (page ??), and other environments may similarly alter their colours or layouts in response to its value.

columns = 1 / 2

Places the document into either **onecolumn** or **twocolumn** mode. The default value is 2.

More advanced column layouts require the user to manually use the **multicols** environment.

font <flag>

If present, activates the font modules, applying the package fonts wherever possible.

This flag is set by default when loading a class, but must be passed explicitly to enable these modules when loading the package directly.

justified <flag>

If present, sets the text to be justified throughout the document. By default, text is left-aligned ('ragged right').

noemph <flag>

If present, the default behaviour of `\emph` is left unchanged. By default, `\emph` is redefined to be an alias of `\RpgKey` (page ??).

nomultitoc <flag>

If present, disables the multi-column table of contents.

oneside <flag>

If present, places the document into one-sided mode. This disables alternating footers, but allows for consistent asymmetric margins (as in this document, where the left margin is enlarged to accommodate macro names).

openright <flag>

If present, places the document into right-opening mode, such that chapters always begin on a right-hand page. By default, chapters may begin on any page (see the documentation of the `book` class for more details).

sections <flag>

If present, activates the sections modules, including the part-page overloads and any theme behaviour which modifies the section commands.

This flag is set by default when loading a class, but must be passed explicitly to enable these modules when loading the package directly.

size = <font-size>pt

Sets the base font size of the document, if permitted by the parent class. The default value is 10pt.

The allowed values depend on the class being used (see the class section, beginning on page ??).

theme = <theme name>

The initial value passed to `\RpgSetTheme` (page ??) once package initialisation is complete.

The default value is `default`, which activates the Default Theme (page ??).

themepath = </path/to/themes/>

Calls `\RpgSetThemePath` (page ??) before the theme is loaded, allowing custom themes to be used.

The default value is `\RpgPackagePath/themes`, with the assumption that a theme called `name` is stored in `themes/name/name.cfg`.

title <flag>

If present, activates the titling module and any theme presets which alter the behaviour of `\maketitle`. This flag is set by default when loading a class, but must be passed explicitly to enable this module when loading the package directly.

Colo(u)rs

RPGTeX defines a number of named colours¹ which are used by its various elements. The default values listed below may be overridden by the active theme.

Co-varying Colours

The following colours are coupled together, and by default all take the same value:

ThemeColor A general-purpose colour from which the other co-varying colours inherit. Default: `white!60!black`.

SidebarColor The background colour of the `RpgSidebar` (page ??) environment.

TableColor The background colour of alternate rows in an `RpgTable` (page ??).

TipColor The background colour of the `RpgTip` (page ??) environment.

Calling `\RpgSetThemeColor` (page ??) assigns a new value to **ThemeColor** and all three co-varying colours simultaneously:

```
\RpgSetThemeColor{blue!20!white}
```

Independent Colours

The remaining colours are set individually:

ContourInnerColor The default colour of the inner text produced by `\RpgContour` (page ??). Default: `black`.

ContourOuterColor The default colour of the outer contour drawn around text by `\RpgContour` (page ??). Default: `white!50!black`.

FiligreeBackColor The background colour of the `RpgFiligree` (page ??) environment. Default: `white`.

FiligreeColor The colour used to draw the filigree frame applied by the `rpgfiligree` style. Default: `black`.

NarrationColor The background colour of the `RpgNarration` (page ??) environment. Default: `F7F2E5` (HTML).

PaperColor The background colour of each page, applied only when the paper background is enabled (see the `bg` option, page 6). Default: `white`.

SecretColor The default frame colour of the `RpgSecret` (page ??) environment. Default: `black!30!white`.

Any of these colours, including the co-varying colours, may be modified individually using the standard `xcolor` interface:

```
\definecolor{NarrationColor}{HTML}{FFFFFFF}  
\colorlet{SidebarColor}{red!10!white}
```

Modifying a co-varying colour in this way changes only that colour, leaving the others unaffected.

¹Although this document uses British spelling, the colour names follow the American spelling used throughout L^AT_EX and `xcolor`, and must be written exactly as given (including capitalisation).

Section Names

Many themes customise their appearance beyond the simple use of `marks` (such as `\chaptermark`). To support this, R_PG_TE_X provides direct access to the names of the current chapter, section, subsection and subsubsection.

Sections Mode Only

These names are recorded by redefining standard L^AT_EX commands, and are therefore only available in *sections mode*. Sections mode is enabled by default when using one of the R_PG_TE_X classes, and may otherwise be enabled by passing the `sections` option to the package (page 6).

When active, sections mode:

- Redefines `\chaptermark`, `\sectionmark`, `\subsectionmark` and `\subsubsectionmark` so that, in addition to their usual behaviour, they store the name they are given in the variables below.
- Sets `secnumdepth` to 0, so that sections and their subdivisions are unnumbered.
- Replaces `\part` with a version which draws a full-page part page using the active theme. The starred form, `\part*`, falls back to the original L^AT_EX command.

Themes may additionally alter the formatting of the sectioning commands when sections mode is active.

Each of the following variables is initially blank, and holds its value until the next command of the same level is called: beginning a new chapter does not, for example, reset `\RpgSectionName`.

`\RpgPartName` {} (*Variable*)

The name given to the last `\part` command.

`\RpgChapterName` {} (*Variable*)

The name given to the last `\chapter` command, or set by calling `\chaptermark`

`\RpgSectionName` {} (*Variable*)

The name given to the last `\section` command, or set by calling `\sectionmark`. Starred sections (`\section*`) do not update this value.

`\RpgSubsectionName` {} (*Variable*)

The name given to the last `\subsection` command, or set by calling `\subsectionmark`. Starred subsections (`\subsection*`) do not update this value.

`\RpgSubsubsectionName` {} (*Variable*)

The name given to the last `\subsubsection` command, or set by calling `\subsubsectionmark`. Starred subsubsections (`\subsubsection*`) do not update this value.

`\RpgGetName` { *part* / *chapter* / *section* / *subsection* / *subsubsection* }

Returns the name stored for the given level (equivalent to the corresponding variable above). Any other input results in an error.

Since these names (other than that of the part) are taken from the `mark` commands, they may be set manually, without creating a new heading, by calling the relevant `mark` command directly.

`\RpgFake` { *level* }{ *name* }

Mimics the creation of a new heading at the given `level` without printing a heading in the document. Specifically, it:

- Adds an entry called `name` to the table of contents at the given level, with a hyperlink anchor at the current position.

- Updates the stored name for that level, as if a real heading had been created. For **chapter** to **subsubsection** this is done by calling the corresponding **mark** command, so any running headers are also updated.

The level must be one of **part**, **chapter**, **section**, **subsection** or **subsubsection**. Any other value results in an error.

```
We are in Chapter '\RpgGetName{chapter}',
section '\RpgGetName{section}'.

\subsubsection{An example subsubsection}

Now I am in '\RpgGetName{subsubsection}'.

\chaptermark{Spoofed Chapter}

I believe I am in '\RpgGetName{chapter}',
though no chapter was actually called.

\chaptermark{Options \& Variables} %% restore
the real value
```

We are in Chapter `Options & Variables', section
`Section Names'.

An example subsubsection

Now I am in `An example subsubsection'.
I believe I am in `Spoofed Chapter', though no chapter
was actually called.

Command Line Interface

By default, $\text{\LaTeX}$ does not have a `command line interface' which allows a user to modify the document from within the command line: changes to the document have to be placed inside the file, and then compiled. However, we found that -- particularly with the *print* option (which suppresses background images on the paper, reducing ink requirements for printing), it was convenient to be able to compile the same document in either `normal' mode, or `print mode', without modifying the text.

PART II

rpgtex For Designers

Index

RpgCMD, 4
RpgChapterName, 9
RpgContour, 8
RpgFake, 9
RpgFiligree, 8
RpgGetName, 9
RpgKey, 7
RpgNarration, 8
RpgPackagePath, 2, 7
RpgPartName, 9
RpgSecret, 8
RpgSectionName, 9
RpgSetThemeColor, 8
RpgSetThemePath, 7
RpgSetTheme, 7
RpgSidebar, 8
RpgSubsectionName, 9
RpgSubsubsectionName, 9
RpgTable, 8
RpgThemeBackgroundColor, 6
RpgThemeFooterDecoration, 6
RpgThemeSetPaper, 6
RpgTip, 8
background, 6
bg, 6
chaptermark, 9
chapter, 9
colorlet, 8
columns, 6
definecolor, 8
edef, 2
emph, 7
font, 7
justified, 7
kpsewhich, 2
maketitle, 8
noemph, 7
nomultitoc, 7
oneside, 7
openright, 7
part*, 9
part, 9
rpglatex, 3
section*, 9
sectionmark, 9
sections, 7
section, 9
size, 7
subsection*, 9
subsectionmark, 9
subsection, 9
subsubsection*, 9
subsubsectionmark, 9
subsubsection, 9
themepath, 7
theme, 7
title, 7
write18, 2